

AVL Trees

SELF-BALANCING LOGIC

The Pinnacle of BST Efficiency in Python

Module 06: Complexity Optimization

Library: CodeHive Data Science

Curriculum Map

01. The Balancing Act	3
02. Understanding the Balance Factor	4
03. Rotation Mechanics	5
04. The Four Failure Cases	6
05. Python Implementation (Part I)	7
06. Python Implementation (Part II)	8
07. Computational Complexity	9
08. The Mathematical Proof of $O(\log n)$	10

01. The Balancing Act

The **AVL Tree** is a type of Binary Search Tree named after its Soviet inventors, **Georgy Adelson-Velsky** and **Evgenii Landis** (1962). It was the first data structure of its kind designed to self-balance.

The Core Problem

A standard BST is only efficient if it is balanced. If data is inserted in sorted order, the BST becomes a "degenerate" tree—effectively a linked list—with $O(n)$ search time. AVL trees solve this by ensuring the height difference between left and right subtrees never exceeds 1.

Key Benefit: By keeping the tree height at a minimum, AVL trees guarantee $O(\log n)$ performance for search, insert, and delete operations, regardless of the order of insertion.

02. The Balance Factor

To determine if a tree is "out of balance," AVL trees calculate a **Balance Factor (BF)** for every node. This factor is derived from the heights of the node's children.

The Equation

$$\text{BF}(X) = \text{height}(\text{rightSubtree}(X)) - \text{height}(\text{leftSubtree}(X))$$

Interpreting the BF

- **0**: The node is perfectly balanced.
- **Positive (>0)**: The node is "Right Heavy."
- **Negative (<0)**: The node is "Left Heavy."

Critical Threshold: If $|\text{BF}| > 1$ (greater than 1 or less than -1), the node is officially unbalanced, and a rotation must be triggered immediately.

03. Rotation Mechanics

Rotations are the fundamental operations that restore balance without violating the BST property (where $\text{Left} < \text{Root} < \text{Right}$).

Left Rotation (RR Case)

Triggered when a node is right-heavy and its right child is also right-heavy. The right child moves up to become the parent, and the original parent becomes its left child.

Right Rotation (LL Case)

Triggered when a node is left-heavy and its left child is also left-heavy. The left child ascends to the parent position, moving the original parent to the right.

Note on Subtrees: During rotation, if the moving child has subtrees of its own, they change parents to maintain the correct in-order traversal and BST ordering.

04. The Four Failure Cases

Depending on where the unbalance occurs, AVL trees apply one of four rotation strategies.

Case	Condition	Necessary Action
Left-Left (LL)	Node is left-heavy, Left Child is left-heavy.	Single Right Rotation
Right-Right (RR)	Node is right-heavy, Right Child is right-heavy.	Single Left Rotation
Left-Right (LR)	Node is left-heavy, Left Child is right-heavy.	Left Rotate Child, then Right Rotate Parent
Right-Left (RL)	Node is right-heavy, Right Child is left-heavy.	Right Rotate Child, then Left Rotate Parent

05. Python Implementation (Part I)

The implementation requires a height attribute on each node and functions to handle the recursive math of rotations.

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
        self.height = 1

def rightRotate(y):
    x = y.left
    T2 = x.right
    # Rotation
    x.right = y
    y.left = T2
    # Update heights
    y.height = 1 + max(getHeight(y.left), getHeight(y.right))
    x.height = 1 + max(getHeight(x.left), getHeight(x.right))
    return x
```

06. Python Implementation (Part II)

The insert function is recursive. After the standard BST insertion, the code "retraces" its steps up the tree to check balance factors and rotate if necessary.

```
def insert(node, data):
    if not node: return TreeNode(data)

    # 1. Standard BST Insert
    if data < node.data: node.left = insert(node.left, data)
    else: node.right = insert(node.right, data)

    # 2. Update Height & Balance
    node.height = 1 + max(getHeight(node.left), getHeight(node.right))
    balance = getBalance(node)

    # 3. Handle 4 Rotation Cases (LL, RR, LR, RL)
    if balance > 1 and getBalance(node.left) ≥ 0:
        return rightRotate(node)
    # ... (Other cases follow similar logic)
    return node
```


07. Computational Complexity

The primary advantage of the AVL tree is its performance consistency. Even in the worst-case scenario, it maintains logarithmic time.

Operation	BST (Unbalanced)	AVL (Self-Balancing)
Search	$O(n)$	$O(\log n)$
Insertion	$O(n)$	$O(\log n)$
Deletion	$O(n)$	$O(\log n)$

Retracing: While rotations add a constant overhead to insertion and deletion, the cost is far outweighed by the search speed gained by keeping h as small as possible.

08. The Mathematical Proof

How do we know the height h is always logarithmic? In a perfect binary tree, the number of nodes n is related to height h by:

$$n = 2^{h+1} - 1$$

Solving for h gives us the time complexity:

1. $n + 1 = 2^{h+1}$
2. $\log_2(n+1) = h + 1$
3. $h = \log_2(n+1) - 1$

In Big O notation, we ignore the constants (+1 and -1). Thus, the height—and the time it takes to find an item—is $O(\log n)$. This means that for a tree with 1,000,000 nodes, the search finishes in about 20 steps.

CodeHive | Conclusion

You have mastered the mechanics of Self-Balancing Trees. Use this knowledge to build faster, more resilient data systems.